Kubernetes CKA 知识点汇总

1. ca-certificates，gnupg，lsb-release 三个包的解释。

答： ca-certificates 软件包提供常用的 SSL 证书，用于在 Linux 系统中进行安全的网络通信。

gnupg 软件包提供一系列用于保护数据安全的加密和解密的工具。

lsb-release 软件包包含了 Linux 发行版相关的信息，例如发行版的名称、版本、代号等。

2. docker-ce，docker-ce-cli，containerd.io，docker-compose-plugin 作用。

答： docker-ce 是 Docker 的社区版本，提供了构建、发布和运行容器的工具和服务。

docker-ce-cli 是 Docker 社区版的命令行工具，用来和 Docker 守护进程进行交互。

containerd.io 是 Docker 运行时的核心组件，提供容器的生命周期、镜像、网络管理等功能。

docker-compose-plugin 用于在单台主机上管理和编排多个容器的运行。

3. K8s 在 1.2 之后就不再支持 docker，请解释对错。

答： 由于 Docker 公司的商业化行为，因此 K8s 官方在 1.2 版本之后就不再直接支持 docker 容器的运行，但仍然可以通过 K8s 提供的 CRI 运行时接口管理 docker 容器。

4. 举例说明创建容器以及以交互方式访问容器的命令？

答：

```
docker run -d -p 8000:80 --name container1 nginx
docker exec -it container1 /bin/bash
```

5. 部署安装 K8s 为什么要关闭 swap 分区？

答： 在虚拟内存管理系统中，尽管 swap 可以在磁盘上增加系统的可用内存量，但在磁盘虚拟页面和内存页面交换过程中会带来额外的磁盘 I/O 负载，加之容器运行环境自身对磁盘的吞吐量有很高的需求，因此，swap 分区的使用会在高负载情况下导致 K8s 系统的整体性能下降并有可能导致应用程序的崩溃。

6. 解释 br_netfilter。

答： br_netfilter 模块用于将桥接流量转发至 iptables 链，如果没有加载 br_netfilter 模块，其并不会影响不同 node 上的 pod 之间的通信，但是会影响相同 node 内的 pod 之间通过 service 实现的通信。

7. 解释 kubeadm，kubectl，kubelet。

答： kubeadm 提供了构建集群的指令，kubeadm init 初始化集群，kubeadm join 扩充集群节点。

kubectl 是 K8s 集群的命令行工具，能够对集群本身进行管理，并进行容器化应用的安装和部署。Kubelet 在集群中每个节点上运行，负责与 AIP-server 进行通信，监控与维护 Pod 中容器的生命周期，同时也负责 CSI（容器存储接口）和 CNI（容器网络接口）的管理。

8. 详细说明 K8s 集群的创建过程。

答：第一步：安装与配置服务器节点，过程略

第二步：在各节点上安装 docker-ce，docker-ce-cli，containerd.io，docker-compose-plugin；部署 CRI-Docker。

第三步：在个节点关闭 Swap 分区；加载 br_netfilter 以允许 iptables 桥接流量；安装 kubeadm，kubectl 和 kubelete 软件包；集成 CRI-Docker。

第四步：在 master 节点上部署集群(kubeadm.yaml)，创建集群(kubeadm init)，加入各节点(kubeadm join)。

第五步：部署 Calico 网络插件，检查验证集群状态。

9. Calico 网络插件在 K8s 集群的作用。

答：Calico 是一个开源网络和网络安全解决方案，适用于容器、虚拟机和基于本地主机的工作负载。Calico 本身是一个三层的虚拟网络方案，它将每个节点都当作路由器（router），将每个节点的容器都当作是“节点路由器”的一个终端并为其分配一个 IP 地址，各节点路由器通过 BGP（Border Gateway Protocol）学习生成路由规则，从而将不同节点上的容器连接起来，从而为 K8s 集群提供了超快的性能和真正的云原生可扩展性，极大地扩展了网络规模和网络边界。

10. kube-apiserver，etcd，kube-controller-manager，kube-scheduler 的作用。

答：kube-apiserver：是的 K8s 的核心控制层，可通过部署更多实例来横向扩展。它对外暴露了 K8s API，是整个集群管理的 API 接口，负责集群内部各个模块之间通信，负责集群安全控制。

etcd：是一致性的 Key/Value 分布式存储，存储了 k8s 集群中所有的元数据，包括各种对象的状态和元信息配置、calico 等网络插件的配置信息。

kube-controller-manager：是处理集群中常规任务的后台线程，当集群资源组件发生各种故障导致系统状态发生变化时，它会通过 API Server 尝试将系统状态修复到“期望状态”。

kube-scheduler：用来监视新创建但尚未分配节点的 Pod，并选择一个节点供其运行。

11. kubelet，kube-proxy 作用。

答：kubelet：运行在每个 node 上，负责与 api-server 通信，并管理本节点的 Pod 及其中的容器。

kube-proxy：运行在每个 node 上，负责监听 service 和 endpoint 的变化情况，并通过 iptables 等来为服务配置负载均衡。

12. 什么是 K8s 的 namespace？

答：在 K8s 中，Namespace（名字空间）提供了一种机制，将同一集群中的资源划分为相互隔离的组，是在多个用户之间划分集群资源的一种方法。

名字空间作用域仅针对带有名字空间的对象，（例如 Deployment、Service 等），这种作用域对集群范围的对象（例如 StorageClass、Node、PersistentVolume 等）不适用。

13. 系统默认创建了哪几个 namespace？

答：default：用户创建资源对象时的默认命名空间。

kube-node-lease：包含用于与各个节点关联的 lease（租约）对象，lease 保证 kubelet 可以向 api-server 发送心跳，以便控制面能够检测到节点故障。

kube-public：所有的客户端（包括未经身份验证的客户端）都可以读取该名字空间。

kube-system：该名字空间用于 Kubernetes 系统创建的对象。

14. 请解释 Pod 是什么？

答：Pod 是 K8s 能够创建、管理和部署的最小计算单元。

Pod 包含一个或多个容器，这些容器共享存储、网络、以及如何运行这些容器的声明。

15. 什么是 Pod 的根容器？

答：每个 Pod 都有一个特殊的被称为“根容器”的 pause 容器，pause 容器有时候也称为 infra 容器，被当作 Pod 中所有容器的“父容器”，pause 在 Pod 中作为共享 Linux Namespace 的基础，同时提供 PID Namespace 共享，为每个 Pod 提供 1 号进程，并收集 Pod 内的僵停进程。

16. 解释 Pod 的生命周期。

答：Pod 遵循预定义的生命周期，起始于 Pending 阶段， 如果至少其中有一个主要容器正常启动，则进入 Running，之后取决于 Pod 中是否有容器以失败状态结束而进入 Succeeded 或者 Failed。

- Pending（挂起）：Pod 已被 K8s 系统接受，但有一个或者多个容器镜像尚未创建。等待时间包括调度 Pod 的时间和通过网络下载镜像的时间。
- Running（运行中）：该 Pod 已经绑定到了一个节点上，Pod 中所有的容器都已被创建，至少有一个容器正在运行，或者正处于启动或重启状态。
- Succeeded（成功）：Pod 中的所有容器都被成功终止，并且不会再重启。
- Failed（失败）：Pod 中的所有容器都已终止了，并且至少有一个容器是因为失败终止，即容器以非 0 状态退出或者被系统终止。
- 未知（Unknown）：因为某些原因无法取得 Pod 的状态，通常是因为与 Pod 所在主机通信失败。

17. Init 类型容器有什么特点，主要用途？

答： Init 容器是一种特殊容器，在 Pod 内的应用容器启动之前运行，Init 容器需要在 Pod 规约中添加 initContainers 字段来声明。Init 容器可以包括一些应用镜像中不存在的实用工具和安装脚本。

18. Sidecar 类型容器和 Init 容器的区别在哪？

答： Sidecar 容器用来提供辅助功能以增强主容器应用，最经典的是日志跟踪，它与主容器并行运行。Init 容器是在主容器启动前提前执行一些准备工作，运行完就处于 pause 状态。

19. 什么是静态 Pod？

答： 静态 Pod 在指定各自节点上由 kubelet 守护进程直接管理的 Pod，它不需要监管，但通过一个镜像 Pod，api-server 任然可以监控静态 Pod 的运行状态。静态 Pod 的名称连字符开头的节点主机名作为后缀。静态 Pod 的创建和销毁方法和由 api-server 创建的 Pod 完全不同。

20. 说明 K8s 控制器的作用？

答： 在 K8s 中，控制器就像 K8s 的大脑，控制器通过监控集群的公共状态来确保集群的当前状态与期望状态保持一致，该期望值就是资源对象的 spec 字段。每个控制器只负责管理某种类型的特定资源。

21. 什么是 ReplicaSet，说明它的主要用途。

答： ReplicaSet 简称 RS，它控制 pod 的副本数始终与预设的值相等，RS 会持续监视这些 Pod 的运行状态，并在 Pod 发生故障时重启 Pod。K8s 官方建议使用 Deployments 代替 RS 来控制 Pod 副本的量，因为 Deployments 支持记录了历史变更的声明式更新，方便进行 RS 的升级和回退等。

22. Deployment 控制器是如何工作的，举例说明其常见用途。

答： 一个 Deployment 能够为 Pod 和 ReplicaSet 提供声明式的更新能力。管理员描述 Deployment 中的目标状态，Deployment 控制器会以可控的进度更改 RS 的实际状态使其变为期望状态。

Deployments 的典型用例有：

- 创建 Deployment 以将 ReplicaSet 上线。
- 通过更新 Deployment 的 PodTemplateSpec，声明 Pod 的新状态 。
- 回滚到较早的 Deployment 版本。
- 扩大 Deployment 规模以承担更多负载。

23. 解释 DaemonSet，列举其使用场景。

答： DaemonSet 能确保集群全部（或者某些）节点上都运行一个 Pod 的副本。当有节点加入时，会为其新增一个 Pod。当有节点从集群移除时，这些 Pod 也会被回收。删除 DaemonSet 将会删除它创建的所有 Pod。

DaemonSet 的一些典型用法：

- 在每个节点上运行集群守护进程。
- 在每个节点上运行日志收集守护进程。
- 在每个节点上运行监控守护进程。

24. 什么是 StatefulSet，其主要作用是什么？

答：StatefulSet 是用来管理有状态应用工作负载的 API 对象。和 Deployment 类似，StatefulSet 可管理基于相同容器规约的一组 Pod。但和 Deployment 不同的是，StatefulSet 为它们的每个 Pod 维护了一个有粘性的 ID，也就是这些 Pod 是基于相同的规约来创建的，但是不能相互替换：无论怎么调度，每个 Pod 都有一个永久不变的 ID。

例如，如果希望使用存储卷为工作负载提供持久存储，就可以引入 StatefulSet 作为解决方案。在这种环境中，StatefulSet 中的单个 Pod 一旦出现故障，则启动新 Pod 替换失败 Pod 时，持久的 Pod 标识符就使得新 Pod 与持久卷的的匹配变得更加容易。

25. 说明 Job 与 CronJob 的功能。

答：Job 负责批量处理短暂的一次性任务，仅执行一次，并保证处理的一个或者多个 Pod 成功结束。CronJob 负责定时任务，在指定的时间周期运行指定的任务，比如备份、生成报表等。

26. Kubernetes 如何在集群的 Pod 之间提供网络服务？

答：Kubernetes 通过一个扁平化的网络模型和 CNI 来为 Pod 提供网络服务，每个 Pod 都有一个独一无二的、可在整个集群内直接访问的 IP 地址，为了实现这个目标，K8s 依赖以下关键机制：

1. 核心网络模型：K8s 规定了所有 Pod 都处于一个统一的、扁平的网络空间中。这意味着：无需 NAT：任何一个 Pod 都可以直接通过 IP 地址访问另一个 Pod，无论它们是否在同一个节点（物理机/虚拟机）上。所见即所得：Pod 看到的自己的 IP 地址，也是其他 Pod 访问它时使用的 IP 地址。
2. 实现方式：CNI，Kubernetes 本身不直接实现这个网络模型，而是定义了一套标准接口，即 CNI(容器网络接口)，它将具体的网络实现任务委托给第三方网络插件如 Calico 或 Flannel。当一个 Pod 被创建时，Kubelet（节点上的代理）会调用已安装的 CNI 插件来完成以下工作：
   - 为 Pod 分配 IP 地址：从一个预定义的 IP 地址池中为新 Pod 分配一个唯一的 IP。
   - 设置网络连接：
     1) 同一节点上的 Pod 通信：CNI 插件通常会创建一个虚拟网桥（如 cbr0），并将节点上所有 Pod 连接到这个网桥上，使它们可以像连接在同一个交换机上一样互相通信。

2）跨节点的 Pod 通信：这是最关键的部分，CNI 插件通过不同技术来实现，最常见的两种是：

- 覆盖网络（Overlay Network）：如 Flannel、Weave Net。它会在各个节点之间创建一个虚拟的隧道（如 VXLAN），将一个节点上 Pod 发出的数据包封装起来，通过节点网络传送到目标 Pod 所在的节点，再解包并交给目标 Pod。这就像在物理网络之上建立了一个“虚拟网络高速公路”。
- 路由（Routing）：如 Calico。它不创建隧道，而是直接配置节点的路由表，每个节点都会被告知“要去往某个 Pod IP 段，应该把数据包发给哪个节点”。这种方式避免了封包和解包的开销，性能更高。

Kubernetes 为每个 Pod 分配一个唯一 IP，然后通过一个可插拔的 CNI 网络插件（如 Calico 或 Flannel）来创建覆盖整个集群的扁平化网络，确保任何 Pod 都可以直接访问其他 Pod。

27. 解释 iptables 和 IPVS 代理模式 Service 的区别。

答： iptables 代理模式的 Service：

- kube-proxy 会监视 K8s 控制节点对 Service 对象和 Endpoints 对象的添加和移除。对每个 Service，它会配置 iptables 规则，从而捕获到达该 Service 的 clusterIP 和端口的请求，进而将请求重定向到 Service 的一组后端中的某个 Pod 上面。对于每个 Endpoints 对象，它也会配置 iptables 规则，这个规则会选择一个后端组合。默认的策略是，kube-proxy 在 iptables 模式下随机选择一个后端。

  使用 iptables 处理流量具有较低的系统开销，因为流量由 Linux netfilter 处理，而无需在用户空间和内核空间之间切换。

IPVS 代理模式的 Service：

- 在 IPVS（IP Virtual Server）模式下，kube-proxy 监视 K8s 服务和端点，并调用 netlink 接口创建相应的 IPVS 规则，并定期将 IPVS 规则与 K8s 服务和端点同步。该控制循环可确保 IPVS 状态与所需状态匹配。访问服务时，IPVS 将流量定向到其中之一的后端 Pod。

  IPVS 代理模式基于类似于 iptables 模式的 netfilter 挂钩函数，但是使用哈希表作为基础数据结构，并且在内核空间中工作，这意味着，与 iptables 模式下的 kube-proxy 相比，IPVS 模式下的 kube-proxy 重定向通信的延迟要短，并且在同步代理规则时具有更好的性能。与其他代理模式相比，IPVS 模式还支持更高的网络流量吞吐量。

28. 举例说明 ClusterIP 类型 Service 的用法。

答： ClusterIP 类型服务发现通过集群的内部 IP 暴露服务，选择该值时服务只能够在集群内部访问。

这也是默认的 ServiceType。

比如，创建服务，对外提供 8000 端口，并把流量引流到具有 app:nginx 的后端 80 端口上。这个场景下，在主机 shell 下是不能直接使用 Pod 的 IP 访问 nginx，需要在 **Pod IP(ClusterIP) 后添加 8000 端口**。

29. 举例说明 NodePort 类型 Service 的用法。

答：NodePort 类型服务发现是通过每个节点上的 IP 和静态端口（NodePort）暴露服务。NodePort 服务会路由到自动创建的 ClusterIP 服务，通过请求<节点 IP>:<节点端口>，就可以从集群的外部访问一个 NodePort 服务。

比如，NodePort 会在节点的特定端口上开通服务，指定的端口为 31788：这个场景下，在**主机的 IP 后添加 31788 端口**就可以访问到 Pod。

30. 举例说明 Headless 类型 Service 的用法。

答：在 Headless 类型的 Service 中，Pod 创建时不会直接返回 Pod 的 Service IP，也就是**不提供 Pod 的 ClusterIP**，而是直接返回众多 Pod 的 IP。这种场景下，需要进入 Pod 内部用集群内的 DNS 进行访问测试。

31. 说明 MetalLB 所实现的 LoadBalancer Service 的原理。

答：MetalLB

在 AWS 或 GCP 等云环境中，当您创建一个 type: LoadBalancer 的 Service 时，云平台会自动为您分配一个公网 IP 并配置好负载均衡器。但在**裸金属（Bare-metal）**或私有数据中心的 Kubernetes 集群中，没有这样的自动化机制。

MetalLB 的核心原理就是：通过在您的本地网络（LAN）上使用标准的网络协议，模拟云厂商的负载均衡器，从而为您的服务分配一个可从外部访问的、稳定的 IP 地址。

它主要通过两种模式来实现这个目标：

1. Layer 2 模式（基于 ARP/NDP）

这是最简单、最通用的模式，它不需要特殊的网络设备。

- **工作原理：**

1. 当一个 LoadBalancer 服务被创建时，MetalLB 从预先配置的 IP 地址池中为它分配一个 IP（例如 192.168.1.100）。
2. 在所有工作节点中，MetalLB 会**选举出一个“领导者”节点**来负责这个 IP。
3. 这个领导者节点上的 speaker 组件（MetalLB 的代理）开始响应针对 192.168.1.100 的 **ARP 请求**（IPv4）或 **NDP 请求**（IPv6）。它会告诉局域网内的路由器：

"192.168.1.100 这个 IP 地址的 MAC 地址是我的！"

4. 因此，所有发往这个 IP 的流量都会被路由器定向到这个领导者节点。

5. 流量到达节点后，再由 Kube-proxy（Kubernetes 的内部网络组件）分发给后端的实际 Pod。

- **故障转移**：如果领导者节点宕机，其他节点的 speaker 会检测到，并选举出一个新的领导者来接管这个 IP 的 ARP 响应，实现高可用。
- **类比**：一个团队对外只有一个电话号码。当电话响起时，团队中只有**一个人（领导者节点）**会接听并处理请求。

2. BGP 模式（基于边界网关协议）

这是更高级、性能和扩展性更好的模式，但要求您的网络设备（如路由器）支持 BGP 协议。

- **工作原理**：

1. 同样，服务被分配一个 IP 地址。
2. 集群中的**所有节点**上的 speaker 组件都会与您的网络路由器建立一个 BGP 会话。
3. 然后，**所有节点**都会通过 BGP 向路由器宣告："要访问 192.168.1.100，可以把流量发给我！"
4. 路由器收到来自多个节点的相同路由宣告后，会认为到达这个 IP 有多条等价路径（ECMP - Equal-Cost Multi-Path）。
5. 因此，路由器会**将流量负载均衡到所有宣告了该路由的节点上**，而不是只发给一个节点。

- **优势**：真正的负载均衡、无单点瓶颈、故障转移更快。
- **类比**：一个团队对外有一个电话号码。当电话响起时，团队中的**每个人（所有节点）**都能接听，电话系统（路由器）会自动将呼叫分配给当前最空闲的人。

核心组件：

- **Controller**：一个 Deployment。负责监听 Service 对象，当发现 type: LoadBalancer 时，从 IP 池中分配一个 IP。
- **Speaker**：一个 DaemonSet，在每个节点上运行。负责执行具体的"宣告"工作，即响应 ARP 请求或通过 BGP 发布路由。

总结要点：

- **解决了什么问题**：让裸金属 K8s 集群也能使用 type: LoadBalancer 服务。
- **核心机制**：通过 speaker 组件，使用**ARP（Layer 2 模式）**或** BGP（BGP 模式）**协议，

在局域网中宣告一个虚拟 IP 的存在，并将其流量拉到集群节点上。

- **模式选择**：Layer 2 模式简单通用；BGP 模式性能更强但需要路由器支持。

一句话总结：MetalLB 通过在每个节点上运行一个"广播员"（Speaker），使用标准的 ARP 或 BGP 协议，将一个虚拟 IP"嫁接"到你的物理网络上，从而巧妙地在裸金属环境里实现了 LoadBalancer 服务。

32. 详细说明 Ingress 的实现原理和它所实现的功能。

答： Ingress 是对集群中服务的外部访问进行管理的 API 对象，典型的访问方式是 HTTP。

Ingress 公开了从集群外部到集群内服务的 HTTP 和 HTTPS 路由，流量路由由 Ingress 资源上定义的规则控制。为了让 Ingress 资源工作，集群必须有一个正在运行的 Ingress 控制器。

Ingress 可以提供负载均衡、SSL 终结和基于名称的虚拟托管。

33. Kubernetes 对集群 Pod 和容器健康状态如何进行监控和检测的。

答： Pod 的健康检查使用存活探针(liveness probes)和就绪性探针(readiness probes)来实现。

34. 解释 LivenessProbes 探针的作用及其适用场景。

答： Liveness Probes（存活探针）用来检测什么时候要重启容器。例如，存活探针可以捕捉到死锁（应用程序在运行，但是无法继续执行后面的步骤）。在这种情况下重启容器有助于让应用程序在有问题的情况下变得可用。

35. 解释 ReadinessProbe 探针的作用及其适用场景。

答： Readiness Probes（就绪探针）可以知道容器什么时候准备好了并可以开始接受请求流量，只有当一个 Pod 内的所有容器都准备好了，才能把这个 Pod 看作就绪。

- 如果要仅在探测成功时才开始向 Pod 发送请求流量，可以指定就绪态探针。
- 如果应用程序对后端服务有严格的依赖性，可以同时实现存活态和就绪态探针。

36. 解释 StartupProbe 探针的作用及其适用场景。

答： Startup Probes（启动探针）可以知道应用程序容器什么时候启动了。用来控制容器在启动成功后再进行存活性和就绪检查，确保这些存活、就绪探针不会影响应用程序的启动。启动探针用于对慢启动容器进行存活性检测，避免它们在启动运行之前就被杀掉。

如果容器需要在启动期间加载大型数据、配置文件或执行迁移，可以使用启动探针。

37. 说明 K8s 中 Pod 级别的 Graceful Shutdown。

答：从 Kubernetes 1.22 开始，terminationGracePeriodSeconds 特性被开启，在杀死容器时，Pod 停止获得新的流量。但在 Pod 中运行的容器不会受到影响，直到超时发生。可以在 Pod 级别或者容器下具体的探针级别设定，探针会优先和覆盖 Pod 级别。

38. 解释什么是 Kubernetes 的 Volume。

答：容器中的文件在磁盘上是临时存放的，这给在其中运行的某些应用程序会带来一些问题，其一是当容器在崩溃时，kubelet 会以干净的状态重启重启容器，原有容器中的文件会丢失。第二个问题是，我们会在同一 Pod 中运行多个容器，而这些容器之间会有共享文件的情况出现。解决这些问题的办法是使用 Kubernetes Volume。

Kubernetes 支持很多类型的卷。Pod 可以同时使用任意数目的卷类型。临时卷类型的生命周期与 Pod 相同，但持久卷可以比 Pod 有更长的存活期。当 Pod 被删除时，临时卷也会被删除，但持久卷会保留在磁盘上。对于给定 Pod 中任何类型的卷，当该 Pod 中的任何容器重启期间 Pod 中的数据都不会丢失。

39. 解释 emptyDir 卷类型的特征。

答：当 Pod 被分派到某个 Node 上时，emptyDir 卷就会被创建，并且在 Pod 运行期间一直会存在。就像其名称表示的那样，emptyDir 卷最初是空的。尽管 Pod 中的容器挂载 emptyDir 卷的路径可能会有不同，但这些容器都可以读写 emptyDir 卷中相同的文件。当 Pod 因为某些原因被从节点上删除时，emptyDir 卷中的数据也会被永久删除。容器崩溃并不会导致 Pod 从节点上移除，因此在容器崩溃期间 emptyDir 卷中的数据是安全的。

40. 解释 hostPath 卷类型的特征。

答：hostPath 卷能将主机节点文件系统上的文件或目录直接挂载到 Pod 中。但要注意的是，要尽可能避免使用这个类型的卷，它会限制 Pod 的迁移性。

41. 解释 PV 卷类型的特征，

答：PV（PersistentVolume，持久卷），是集群中的一块存储，可以由管理员事先供应，或者使用存储类（Storage Class）来动态供应。PV 持久卷和节点一样，都是集群资源，PV 持久卷和普通的 Volume 类似，也是使用卷插件来实现的，只是它拥有独立于任何使用 PV 的 Pod 的生命周期。

42. 什么是 PVC，如何使用它。

答：PVC（PersistentVolumeClaim，持久卷申领）表达的是用户对存储的请求。概念上与 Pod 类似。

Pod 会耗用节点资源，而 PVC 申领会耗用 PV 资源。Pod 可以请求特定数量的资源（CPU 和内存）；同样 PVC 申领也可以请求特定的大小和访问模式。

Pod 将 PVC 申领当做存储卷来使用。集群会检视 PVC 申领，找到所绑定的卷，并为 Pod 挂载该卷。对于支持多种访问模式的卷，用户要在 Pod 中以卷的形式使用申领时指定期望的访问模式。一旦用户有了申领对象并且该申领已经被绑定，则所绑定的 PV 卷在用户仍然需要它期间一直属于该用户。

43. PV 有哪几种访问模式，详细说明。

答：PV 的访问模式有四种：

- ReadWriteOnce：卷可以被一个节点以读写方式挂载。该访问模式也允许运行在同一节点上的多个 Pod 访问卷。
- ReadOnlyMany：卷可以被多个节点以只读方式挂载。
- ReadWriteMany：卷可以被多个节点以读写方式挂载。
- ReadWriteOncePod：卷可以被单个 Pod 以读写方式挂载。

44. 解释 PV 的回收策略。

答：PV 的回收策略有：

- Retain：需要手动回收
- Recycle：执行的基本操作是 rm -rf /thevolume/*
- Delete：对于支持 Delete 回收策略的卷插件，删除动作会将 PersistentVolume 对象从 K8s 中移除，同时也会从外部基础设施（如 AWS EBS 或 GCE PD 卷）中移除所关联的存储资源。

45. 如何将特定 Pod 调度到指定的节点？

答：可以使用下列方法中的任何一种来选择 K8s 对特定 Pod 的调度：

- 与节点标签匹配的 nodeSelector
- 亲和性与反亲和性
- nodeName 字段
- Pod 拓扑分布约束

46. 什么是节点的亲和性？

答：节点亲和性（Node Affinity）是 Pod 的一种属性，它使 Pod 被吸引到一类特定的节点（可能出于一种偏好，也可能是硬性要求）。节点亲和性概念上类似于 nodeSelector，可以根据节点上的标签来约束 Pod 可以调度到哪些节点上。

47. 什么是污点，它的主要用途是什么？

答：与节点亲和性不同，污点（Taint）使得节点能够排斥一类特定的 Pod。

污点和容忍度相互配合，可以用来避免 Pod 被分配到不合适的节点上。每个节点上都可以应用一个或多个污点，这意味着对于那些不能容忍这些污点的 Pod，是不会被该节点接受的。

容忍度（Toleration）是应用于 Pod 上的。容忍度允许调度器调度带有对应污点的 Pod。容忍度允许调度但并不保证调度：作为其功能的一部分，调度器也会评估其他参数。

48. 解释 ConfigMap 的作用。

答：ConfigMap 是一个 API 对象，用来将非机密性的数据保存到键值对中。使用时，Pod 可以将其用作环境变量、命令行参数或者存储卷中的配置文件。使用 ConfigMap 来将应用的配置数据和应用程序代码分开。ConfigMap 中保存的数据不可超过 1MiB。

49. Secret 和 ConfigMap 相比较有哪些优点。

答：Secret 的功能类似于 ConfigMap，但被设计专门用于保存机密数据，例如密码、令牌或密钥对象。使用 Secret 意味着可以不需要在应用程序代码中包含机密数据，用以增强此类数据的安全性。

50. 解释 ResourceQuota 的作用。

答：ResourceQuota（资源配额）用来在有多个用户或团队共享具有固定节点数目的集群时，可以解决有人使用超过其基于公平原则所分配到的资源量的问题。

通过 ResourceQuota 对象可以对每个命名空间的资源消耗总量提供限制。它可以限制命名空间中某种类型的对象的总数目上限，也可以限制命名空间中的 Pod 可以使用的计算资源的总上限。

51. 解释 Service Account 的用途。

答：Service Account（服务账号）为 Pod 中运行的进程提供身份标识，并映射到 ServiceAccount 对象。当应用向 API 服务器执行身份认证时，应用会将自己标识为某个用户，K8s 能够识别用户的概念，但是 K8s 自身并不提供 User API。因此，Service Account 是为了方便 Pod 里面的进程调用 K8s API 或其他外部服务而设计的。

52. 详细解释 Role 和 ClusterRole。

答：基于角色（Role）的访问控制（RBAC）是一种基于组织中用户的角色来调节控制对计算机或网络资源的访问的方法。RBAC API 声明了四种 K8s 对象：Role、ClusterRole、RoleBinding 和 ClusterRoleBinding.

Role 总是用来在某个命名空间内设置访问权限，在创建 Role 时，必须指定该 Role 所属的名字空间。ClusterRole 则是一个集群作用域的资源。这两种资源的名字不同（Role 和 ClusterRole）是因为 K8s 对象要么是名字空间作用域的，要么是集群作用域的，不可两者兼具。

如果希望在命名空间内定义角色，就应该使用 Role，如果希望定义集群范围的角色，则应该使用 ClusterRole。

53. 什么是 K8s 的 NetworkPolicy？

答：NetworkPolicy（网络策略）是一种以应用为中心的结构，允许我们设置如何允许 Pod 与网络上的各类网络“实体”通信。NetworkPolicy 适用于一端或两端与 Pod 的连接，与其他连接无关。

网络策略对 Pod 间的流量控制是通过如下三个标识符的组合来辩识的：

- 其他被允许的 Pods（例外：Pod 无法阻塞对自身的访问）。
- 被允许的名字空间。
- IP 组块（例外：与 Pod 运行所在的节点的通信总是被允许的，无论 Pod 或节点的 IP 地址）。

54. 详细描述在 K8s 中如何控制跨 Namespace 的 Pod 访问？

答：默认情况下，Pod 是非隔离的，它们接受任何来源的流量。

Pod 在被某 NetworkPolicy 选中时进入被隔离状态，一旦名字空间中有 NetworkPolicy 选择了特定的 Pod，则该 Pod 就会拒绝该 NetworkPolicy 所不允许的连接。

网络策略不会冲突，它们是累积的。如果任何一个或多个策略选择了一个 Pod，则该 Pod 受限于这些策略的入站（Ingress）/出站（Egress）规则的并集，评估的顺序并不会影响策略的结果。

为了允许两个 Pods 之间的网络数据流，源端 Pod 上的出站规则和目标端 Pod 上的入站规则都需要允许该流量。如果源端的出站规则或目标端的入站规则拒绝该流量，则流量将被拒绝。

55. 举例说明 K8s 中都有哪些常规的维护管理操作。

答：常见的维护管理操作有：

- 查看 pod 详情，（可以通过最下面的 event 查看事件）

```
# kubectl describe pod frontend
```

- 查看 pod 日志

```
# kubectl logs frontend
```

- 查看 pod 资源用量

```
# kubectl top pod frontend
```

- 查看节点资源用量

```
# kubectl top nodes
```

- 禁止 pod 调度

```
# kubectl cordon host1
```

- 驱逐现有 pod

```
# kubectl drain master --ignore-daemonsets
```

56. 如何升级 K8s 到新的版本？在升级过程中应该注意哪些事项？

答：升级工作的基本流程如下：

1.确定要升级到哪个版本 apt list；2.驱逐 Master 节点上的现有任务；

3.安装目标的 kubeadm、kubelet、kubectl；4.验证升级计划；

5.执行升级；6.恢复调度功能；7.验证集群状态和版本

注意事项有：

- 务必仔细阅读发行说明。
- 集群应使用静态控制平面和 etcd pod 或外部 etcd。
- 确保备份所有重要组件，例如存储在数据库中的应用程序级状态。

57. 解释 ETCD 及其备份和恢复的过程。

答：etcd 是一个一致且高度可用的键值存储，用作 K8s 所有集群数据的后备存储。

备份过程：申明 etcdapi 版本；完成备份；验证备份有效性。

恢复过程：停止 kubelet 服务；恢复 etcd；启动 kubelet 服务；查询数据库健康状态。

58. Kustomization 在 Kubernetes 中的作用。

答：Kustomize 是 Kubernetes 的原生配置管理工具，它的核心作用是：以一种无模板（template-free）的方式，对 YAML 配置文件进行自定义和管理，它解决了在不同环境（如开发、测试、生产）中管理大量相似但又有细微差别的 YAML 文件的难题。

Kustomize 的工作方式采用“基础（Base）+ 覆盖（Overlay）”的模式：

1. 定义一个 Base:包含一套通用的、在所有环境中都共享的 YAML 文件(如 Deployment.yaml, Service.yaml)。
2. 创建一个 Overlay：针对特定环境（如生产环境），只定义与“基础”配置不同的部分。这些不同点被称为“补丁”（Patch）。例如：
   - 增加副本数量 (replicas)
   - 更改镜像标签 (image tag)
   - 添加特定的标签 (labels) 或注解 (annotations)
   - 修改 ConfigMap 中的配置数据
3. 使用 kustomization.yaml 文件：这个文件是核心，它会告诉 Kustomize 基础配置在哪里、需要应用哪些补丁或修改。

当运行 kubectl apply -k <目录> 时，Kustomize 会动态地将 Base 和 Overlay 合并，生成最终的 YAML 配置并应用到集群中。

Kustomize 通过 Base + Overlay 的模式，让我们能高效方便地管理多环境的 Kubernetes 配置，Kustomize 是官方推荐的、轻量级的配置管理方式。

59. 什么是 CRD，它对 K8s 有什么重要意义？

答：CRD (Custom Resource Definition)，即“自定义资源定义”。

CRD 允许用户扩展 Kubernetes API，创建和 Pod、Deployment 一样的、属于我们自己的、全新的资源类型。CRD 定义了新资源的名称、结构和规范，是一个“模板”或“蓝图”；CR(Custom Resource)：是根据这个“模板”创建的具体实例。

CRD 是 Kubernetes 能够成为云原生“操作系统”的基石，其重要意义主要体现在以下几点：

1. 极强的可扩展性：CRD 将 Kubernetes 从一个“容器编排平台”变成了一个 “通用控制平台”。任何系统（数据库、消息队列、监控等）都可以通过 CRD 被 K8s 统一管理。
2. 实现声明式 API：可以像管理 Pod 一样，用 YAML 文件来声明性地管理你的自定义应用。
3. 催生了 Operator 模式：CRD 是实现 Operator 模式的核心。Operator 是一个自定义控制器，它会“监听”我们创建的自定义资源（CR），并根据其状态（例如“需要 3 个数据库副本”）执行具体操作，确保系统的实际状态与声明的期望状态一致。
4. 构建了丰富的生态系统：几乎所有复杂的云原生项目（如 Istio、ArgoCD 等）都基于 CRD 构建，它们将自身功能无缝集成到 K8s 中，让用户可以用统一的 kubectl 命令来管理一切。

总之，CRD 让 Kubernetes 变得无所不能。它把 K8s 变成了一个可以构建平台（Platform for building platforms）的平台，是 K8s 强大生命力和生态系统的关键。

60. CRD 的典型应用场景有哪些，举例说明。

答：CRD 的典型应用场景主要有：

1. 数据库管理：如 MySQL、Redis 等数据库实例的声明式管理。
2. 应用部署：自定义应用部署资源（如 Application CRD）。
3. 监控告警：Prometheus 的 ServiceMonitor、AlertRule 等。
4. 网络策略：自定义网络配置和安全策略。
5. 备份恢复：定时备份任务的声明式定义。
6. CI/CD 流水线：自定义 Pipeline 资源。

61. 结合教材，解释 CRD 的完整创建和使用过程。

答：CRD 的完整创建和使用过程包括：

1. 定义 CRD：创建 CRD YAML，定义资源结构和验证规则。
2. 部署 CRD：kubectl apply -f my-crd.yaml。

3. 创建 Controller：实现自定义控制器监听 CR 事件。

4. 部署 Controller：部署控制器 Pod。

5. 使用 CR：创建自定义资源实例

62. Gateway API 有哪几个核心组件，分别有哪些作用？

答： Gateway API 是 Kubernetes 新一代的流量管理接口，其核心是通过角色分离的设计，让不同团队（基础设施、集群运维、应用开发）能够独立且安全地管理流量。它主要有三个核心组件：

1. GatewayClass (网关类)：定义了集群中哪种负载均衡器可用，以及由哪个控制器（如 Nginx, Istio 等）来管理，它是一个集群级别的、全局性的资源。
2. Gateway (网关)：是根据 GatewayClass 模板在集群中创建的一个实际流量入口点，它配置了监听的端口（如 80, 443）、协议、TLS 证书，并决定允许哪些命名空间的应用（通过 Route）附加到它上面。
3. Route (路由)：将来自 Gateway 的流量，根据匹配规则（如主机名、路径、请求头），转发到具体的后端服务（Service），它是应用开发者定义服务如何暴露的地方。

这个模型清晰地划分了职责：GatewayClass (模板) → Gateway (实例) → HTTPRoute (规则)

1. 集群管理员提供 GatewayClass (比如，我们支持 Nginx 网关)。
2. 运维团队创建一个 Gateway (比如，在生产环境创建一个监听 443 端口的 Nginx 网关实例)。
3. 应用开发者创建一个 HTTPRoute (比如，将 api.myapp.com/v1 的流量路由到我的 v1-service)。

这种分离使得网络管理更加安全、灵活且易于扩展，是它相对于传统 Ingress 的最大优势。

63. 说明 PriorityClass 在解决 K8s 资源争用时的调度决策原理。

答： PriorityClass 是 Kubernetes 中定义 Pod 重要性级别的机制。它的核心原理是：为 Pod 分配一个优先级分值，K8s 调度器（kube-scheduler）在资源紧张时会基于这个分值做出两个关键决策：调度（Scheduling）和抢占（Preemption）。

1. 调度决策：高优先级 Pod 优先“入座” (Pod Queuing)

   当集群中有可用的资源，但有多个 Pod 都在等待被调度时，调度器会查看它们的优先级。

   这就像一个等待队列，高优先级的 Pod 可以“插队”到前面，优先获得可用的 CPU 和内存资源。

   普通优先级的 Pod 需要等高优先级的 Pod 都调度完成后，才能获得机会。

2. 抢占决策：高优先级 Pod 可以“抢占”座位 (Pod Preemption)

   当一个高优先级的 Pod 需要被调度，但集群中没有任何节点有足够的空闲资源时，抢占机制就会被触发。例如：当一个非常重要的“VIP” Pod（高优先级）想要入场，但所有座位都满了，调

度器会找到一个坐着“普通观众” Pod（低优先级）的座位，通知他离开，然后让“VIP”入座。

工作流程简介：

1. 管理员定义 PriorityClass：创建一个全局的 PriorityClass 对象，并为其设置一个整数 value（值越大，优先级越高）。
2. Pod 引用 PriorityClass：在 Pod 的配置中，通过 priorityClassName 字段引用这个 PriorityClass。
3. 调度器执行决策：kube-scheduler 在调度时读取 Pod 的优先级，并根据上述“排队优先”和“抢占驱逐”的原则进行决策。

通过 PriorityClass 这种机制，K8s 确保了无论集群多忙、资源多紧张，系统最重要的核心应用（比如数据库、支付网关等）总能被优先执行，从而保证了整个系统的可用性和稳定性。

64. HPA 是如何在 K8s 集群中实现自动扩缩容机制的？

答：HPA（Horizontal Pod Autoscaler）的实现原理就像一个自动恒温器。它通过一个持续的“监控-比较-调整”控制循环过程，来自动增减 Pod 的副本数量，以匹配实际的负载需求。

HPA 机制主要依赖三个核心组件：

1. 指标服务器 (Metrics Server)：作为数据来源，它是一个轻量级的集群插件，负责从每个节点（Kubelet）收集所有 Pod 的资源使用指标（主要是 CPU 和内存），并通过 Kubernetes 的 Metrics API 提供给其他组件查询。
2. HPA 控制器 (HPA Controller)：这是自动扩缩容的 “大脑”和决策者，它内置于 K8s 的 controller-manager 中，通常每 15 秒一次执行获取指标、计算期望值和做出决策。
3. 目标资源 (如 Deployment, StatefulSet)：当 HPA 控制器做出决策后，它不会自己去创建或删除 Pod，它只是去更新目标资源（比如一个 Deployment）的 .spec.replicas 字段，即“期望的副本数”。Deployment 控制器（或其他控制器）会一直监听这个字段，一旦发现期望副本数变化了，它就会立即行动，创建出新的 Pod（扩容）或终止多余的 Pod（缩容），直到实际运行的 Pod 数量与期望值一致。

HPA 通过 HPA 控制器持续监控 Metrics Server 提供的 Pod 负载数据，并与用户设定的目标值比较，然后自动调整 Deployment 的副本数，确保系统负载始终保持在一个健康的水平。

65. 请谈谈 Argo CD 的核心架构组件有哪些，分别做简要说明。

答：Argo CD 是一个为 Kubernetes 设计的声明式 GitOps 持续交付工具，其核心架构由以下几个关键组件构成，它们协同工作，将 Git 仓库中的配置状态同步到 Kubernetes 集群中。

2. API Server (API 服务器)：这是一个 gRPC/REST 服务器，负责对外提供 API 服务。Argo

CD 的 Web UI、CLI 以及 CI/CD 系统都是通过它来与 Argo CD 进行交互的，它处理所有外部请求，如用户认证、应用创建/删除、获取应用状态、触发同步等。

3. Repository Server（仓库服务器）：这是一个内部服务，专门负责与 Git 仓库打交道，它的核心任务是：
   - 克隆指定的 Git 仓库
   - 生成最终的 YAML 清单：如果仓库中使用的是 Helm、Kustomize 或 Jsonnet 等模板工具，它会负责渲染这些模板，生成标准的 Kubernetes YAML 文件。
   - 缓存仓库内容，避免频繁地从 Git 服务器拉取，提高性能。
4. Application Controller（应用控制器）：这是实现 GitOps 核心逻辑的控制器，它是一个标准的 Kubernetes 控制器，持续执行以下“协调循环”（Reconciliation Loop）：
   - 监控：监听在 Kubernetes 中创建的 Application 自定义资源（CRD）。
   - 对比：将 Git 仓库中的目标状态（由 Repository Server 提供）与 Kubernetes 集群中的实际运行状态进行比较。
   - 报告：根据对比结果，更新 Application 资源的状态，标记为 Synced（已同步）或 OutOfSync（未同步）。
   - 执行（可选）：如果配置了自动同步，当发现状态不一致时，它会自动调用 Kubernetes API 来创建、更新或删除资源，使集群的实际状态与 Git 中的目标状态保持一致。
5. Redis（缓存）：Redis 在 Argo CD 中至关重要，它被用来缓存所有与应用状态相关的数据：
   - 从 Repository Server 获取的 Git 清单缓存。
   - 从 Kubernetes API Server 获取的集群实时状态缓存。
   - API Server 的请求结果缓存。

用户通过 API Server 定义一个 Application，Application Controller 指挥 Repository Server 从 Git 获取配置，然后持续对比并同步到 Kubernetes 集群，而 Redis 则为整个过程提供高速缓存支持。

66. 你是如何理解 CI/CD 流水线技术的，它和提升企业竞争力有什么关系？

答：CI/CD 的基本概念：

1. 持续集成（Continuous Integration, CI）：指开发者频繁地将代码变更合并到主分支，并通过自动化构建和测试来验证这些变更。
2. 持续交付（Continuous Delivery, CD）：在持续集成的基础上，进一步将代码自动部署到预生产环境或生产环境中，确保代码随时可以发布。

3. 持续部署 (Continuous Deployment, CD)：是持续交付的进一步延伸，它会自动将代码变更部署到生产环境，无需人工干预。

一个典型的 CI/CD 流水线包括以下几个阶段：

1. 代码提交：开发者将代码提交到版本控制系统（如 Gitee）。
2. 构建：自动化工具（如 Argo CD, Jenkins, GitLab CI 等）检测到代码变更后，触发构建过程，编译代码并生成可执行文件或包。
3. 测试：自动运行单元测试、集成测试、端到端测试等，确保代码质量。
4. 部署：如果测试通过，代码会被自动部署到预生产环境或生产环境。
5. 监控和反馈：部署后，系统会自动监控应用性能和用户反馈，以便及时发现和解决问题。

CI/CD 流水线技术通过自动化和标准化的流程，显著提高了软件开发、测试和部署的效率。在提高了代码质量和开发效率的同时，还加快了产品上市时间，降低了风险，增强了团队协作，最终提升了企业的竞争力。在当今快速变化的市场环境中，CI/CD 成为了企业保持竞争优势的重要工具。

THE END.